

Software Architecture Report

SPA vs MPA, Project Decisions, and REST vs gRPC

1 Difference Between Multiple Page Application (MPA) and Single Page Application (SPA)

Multiple Page Application (MPA)

An **MPA** is a traditional web application where each user action (click, submit, navigation) requests a **new page from the server**.

Characteristics:

- Full page reload on every request
- Rendering is mostly handled on the server
- Simple architecture (HTML + backend templates)
- SEO-friendly by default

Examples:

- News websites
 - E-commerce platforms (traditional ones)
-

Single Page Application (SPA)

An **SPA** loads a single HTML page initially, and then dynamically updates content using JavaScript without reloading the page.

Characteristics:

- No full page reload (smooth UX)
- Heavy use of JavaScript frameworks (React, Angular, Vue)
- Communicates with backend via APIs (AJAX / Fetch)
- Faster after initial load

Examples:

- Gmail
 - Facebook
 - Dashboards
-

Key Differences

Feature	MPA	SPA
Page Reload	Yes	No
Performance	Slower navigation	Faster navigation

Feature	MPA	SPA
SEO	Better	Needs optimization
Complexity	Simple	More complex
Backend Role	High	Mostly API-based

2 Choosing the Best Approach for Projects

Let's assume the three recommended projects are:

Project 1: Student Management System (CRUD - MVC)

Recommended:  MPA

Why:

- Mostly forms (Add / Edit / Delete)
- No need for dynamic UI updates
- ASP.NET MVC works naturally as MPA
- Easier to implement and debug

 Conclusion:

MPA is better because the system is **data-driven, simple, and doesn't need real-time interaction**

Project 2: Analytics Dashboard (Charts, Real-time Data)

Recommended:  SPA

Why:

- Requires dynamic updates (charts, filters)
- Better user experience without reload
- Can fetch data asynchronously
- Works well with APIs

 Conclusion:

SPA is better because it provides **smooth interaction and real-time updates**

Project 3: AI-Based Web App (e.g., Image Detection / YOLO)

Recommended:  SPA (or Hybrid SPA)

Why:

- Needs async file upload and result display
- Might show progress/loading states
- Better UX when interacting with AI results
- Can call backend API (Python / Flask / FastAPI)

 Conclusion:

SPA is better because AI apps need **interactive UI and asynchronous processing**

🔥 Final Comparison for Projects

Project	Type	Best Approach	Reason
Student System	CRUD	MPA	Simple & form-based
Dashboard	Data Visualization	SPA	Dynamic & real-time
AI App	Interactive	SPA	Async + better UX

3 Difference Between REST and gRPC

◆ REST (Representational State Transfer)

REST is an architectural style that uses **HTTP requests** to communicate between client and server.

✅ Characteristics:

- Uses HTTP methods (GET, POST, PUT, DELETE)
- Data format: JSON (mostly)
- Human-readable
- Widely used in web apps

✅ Example:

```
GET /api/students/1
```

◆ gRPC (Google Remote Procedure Call)

gRPC is a high-performance RPC framework that uses **Protocol Buffers (Protobuf)** instead of JSON.

✅ Characteristics:

- Uses HTTP/2
- Faster and more efficient
- Strongly typed contracts (.proto files)
- Supports streaming

✅ Example:

```
service StudentService {  
  rpc GetStudent (StudentRequest) returns (StudentResponse);  
}
```

🔥 Key Differences

Feature	REST	gRPC
---------	------	------

Protocol	HTTP/1.1	HTTP/2
Data Format	JSON	Protobuf (binary)
Speed	Slower	Faster
Readability	Human-readable	Not readable
Streaming	Limited	Built-in
Use Case	Web apps	Microservices / high-performance systems

When to Use Each?

✅ Use REST when:

- Building web APIs for browsers
- Need simplicity and readability
- Public APIs

✅ Use gRPC when:

- Microservices communication
- High performance is required
- Internal systems

🏁 Conclusion

- **MPA vs SPA:**
 - MPA is simple and suitable for CRUD systems
 - SPA is powerful and ideal for dynamic and interactive apps
- **Project Decisions:**
 - Use MPA for simple systems
 - Use SPA for dashboards and AI-based apps
- **REST vs gRPC:**
 - REST is easier and widely used
 - gRPC is faster and better for internal high-performance systems